IILM UNIVERSITY, GREATER NOIDA

Department of Computer Science & Engineering | Summer Internship Evaluation (AY 2026-27)

FINAL YEAR INTERNSHIP PRESENTATION

Cloud Infrastructure Automation & Containerized Deployment Platform

Production Architecture • AWS Infrastructure as Code • Docker • Kubernetes • Jenkins CI/CD • Observability

STUDENT CREDENTIALS

Student Name: PAWAN DUBEY
Roll Number: CS-2341492
Program & Sem: B.Tech CSE (Cloud Computing), 7th Sem
Section: 4CSE8
University Email: pawan.dubey.cs27@iilm.edu

ORGANIZATION & EVALUATION

Organization Name: SystemaOps (systemaops.com)
Internship Role: Cloud & DevOps Engineering Intern
Company Email: pawan.dubey@systemaops.in
Duration: 3 Months (Summer 2026)
Faculty Coordinator: Ms. Surabhi Purwar

Executive Summary & Internship Context

100%

Infrastructure Automation

Provisioned AWS Multi-AZ VPC, ALB & RDS via HashiCorp Terraform modules

99.99%

High Availability Uptime

Zero-downtime rolling container deployments managed on Kubernetes cluster

50%+

Reduction in MTTR

Prometheus time-series metric scrapers paired with Grafana dashboard alerts

Organization Profile & Engineering Internship Scope

SystemaOps (systemaops.com)

- Premier Cloud Engineering & DevSecOps consultancy
- Enables enterprise cloud transformations & microservices migration
- Expertise in HashiCorp Terraform, AWS Architecture, and Kubernetes SRE
- Focus on security compliance, cost optimization, and disaster recovery

DevOps Engineering Intern Responsibilities

- Formulated declarative Terraform modules for Multi-AZ AWS VPC & ALB
- Containerized microservices stack with Docker & Docker Compose
- Built automated CI/CD pipelines in Jenkins and GitHub Actions
- Configured Kubernetes rolling updates & Nginx reverse proxy ingress
- Deployed Prometheus & Grafana monitoring dashboards for operational metrics

Problem Statement & Strategic Engineering Objectives

Legacy System Bottlenecks

❌ Environmental Inconsistency: 'Works on my machine' defects due to unstandardized environments.

❌ Manual Cloud Configuration: Slow, error-prone manual server and database setups leading to configuration drift.

❌ Release Downtime: Service interruptions during deployment release cycles.

❌ Lack of Central Telemetry: Poor visibility into microservices resource utilization and live latency spikes.

Project Technical Objectives

✅ Infrastructure as Code: Automate 100% AWS VPC, EC2, and RDS provisioning using Terraform.

✅ Containerization: Standardize microservices with multi-stage Docker builds.

✅ Automated CI/CD: Implement zero-downtime Jenkins & GitHub Actions deployment pipelines.

✅ Observability: Real-time Prometheus metrics collection & Grafana dashboard alerts.

Multi-AZ AWS 3-Tier Production System Architecture

Tier 1: Public Presentation Tier (AWS Load Balancing)

AWS Application Load Balancer (ALB) distributed across Public Subnets (Multi-AZ)

Nginx Reverse Proxy • TLS/SSL Termination • Ingress Routing • Port 80/443 External Traffic

Tier 2: Private Application Tier (EC2 & Kubernetes Cluster)

EC2 Auto-Scaling Group & Kubernetes Worker Nodes in Isolated Private Subnets

Container Microservices • Port 3000/5000 • Outbound Internet via Secure AWS NAT Gateways

Tier 3: Private Data Tier (Amazon RDS & Persistence)

Amazon Relational Database Service (RDS - Multi-AZ MongoDB/PostgreSQL)

Isolated Private Database Subnets • Multi-AZ Storage Replication • Strict SG Ingress Rules

Enterprise DevOps Stack & Infrastructure as Code (Terraform)

AWS Cloud Infrastructure

VPC, EC2, ALB, S3, RDS, CloudWatch, NAT Gateways

HashiCorp Terraform

Declarative IaC modules & S3 remote state file locking

Docker & Compose

Container packaging, multi-stage lightweight builds

Kubernetes & Helm

Pod auto-scaling, self-healing & package management

Jenkins & GitHub Actions

Automated CI/CD build, test & deployment pipelines

Prometheus & Grafana

Time-series metric collection & analytical dashboards

Container Orchestration & Kubernetes Deployment

Docker Multi-Stage Containerization

- Multi-stage Dockerfiles separating build dependencies from production runtime
- Container image size optimized from 850 MB to 180 MB (> 75% reduction)
- Docker Compose orchestration for local developer environment replication
- Environment variables secured via Docker Secrets & .env templates

Kubernetes & Helm Orchestration

- Kubernetes Deployment manifests with rolling update strategy (maxSurge=1, maxUnavailable=0)
- Horizontal Pod Autoscaler (HPA) configured for CPU utilization > 70%
- Liveness and Readiness probes ensuring self-healing pod recovery
- Nginx Ingress Controller managing SSL termination & path-based routing

Automated CI/CD Pipeline Workflow (Jenkins & GitHub Actions)

Stage 1	Stage 2	Stage 3	Stage 4	Stage 5
Step 1: Commit	**Step 2: Test & Scan**	**Step 3: Build Image**	**Step 4: Push Registry**	**Step 5: Deployment**
GitHub Push / PR Trigger	Jest Unit Tests & Trivy Security Scan	Docker Multi-stage Containerization	Amazon ECR Tagging & Image Push	Zero-Downtime Rolling K8s Rollout

Operational Observability, Metrics & Security Control

Prometheus & Grafana Observability

- Prometheus time-series scrapers scraping node & cluster targets
- Node Exporter & kube-state-metrics operational tracking
- Real-time Grafana dashboards displaying CPU, Memory, and Latency
- Alertmanager sending instant Slack/Email alerts for HTTP 5xx spikes

Cloud Security & Governance

- AWS IAM Roles enforcing Least Privilege policies across all nodes
- Isolated Private Subnets for Amazon RDS (zero public IP exposure)
- Nginx SSL/TLS Termination with TLS 1.3 encryption protocols
- JWT Authentication & Role-Based Access Control (RBAC) in application tier

CONCLUSION & ENGINEERING OUTCOMES

- ✓ Mastered enterprise Cloud & DevOps engineering workflows during 3-Month Summer Internship at SystemaOps.
- ✓ Successfully delivered production-grade Multi-AZ AWS Infrastructure as Code (Terraform), CI/CD, and Observability.
- ✓ Achieved 100% infrastructure automation, zero-downtime rolling deployments, and 50%+ reduction in MTTR.
- ✓ All code artifacts, report PDF, presentation PDF, and certificate PDF pushed to public GitHub repository.

Thank You! Questions & Discussion